Übungsblatt 2: RISC-V-Registerbefehle

Mikrocomputertechnik 1

Aufgabe 1: Eine Registerspur (8 Punkte)

RV32I, 32-Bit-Register. Anfang `t0=t1=t2=0`; `t0=x5`, `t1=x6`, `t2=x7`.

```
addi t0, x0, 12
addi t1, x0, -5
sub  t2, t0, t1
add  t0, t2, t1
xori t1, t0, -1
add  x0, t2, t0
```

1. Erstellen Sie eine Tabelle mit `t0`, `t1`, `t2` und `x0` nach jeder Zeile. (4 P)
2. Welche alten Quellen liest Zeile 4? (1 P)
3. Welchen Wert zeigt `x6` nach Zeile 5 und weshalb? (1 P)
4. Welcher Rechenwert entsteht in Zeile 6, und welche Register ändern sich? (2 P)

Aufgabe 2: Umlauf und Masken (10 Punkte)

1. Bestimmen Sie zu jeder Addition Ergebniswort, signed Ergebnis, hinausfallenden Carry und signed Überlauf. Gespeichert werden 32 Bit. (6 P)

Quelle A	Quelle B
`0xFFFFFFFE`	`5`
`0x60000000`	`0x30000000`
`0x90000000`	`0x90000000`

2. Für `t0=0x0000006C`, `t1=0x00000033`: Bestimmen Sie `t2`, `t3` und `t4`. (3 P)

```
and t2, t0, t1
or  t3, t0, t1
xor t4, t0, t1
```

3. Weshalb liefert `xor` hier keine normalisierte Wahrheitszahl? (1 P)

Aufgabe 3: Schiebeweiten prüfen (8 Punkte)

Jede Zeile startet unabhängig mit `t0=0xFFFFFFF5` (−11), `t1=2`, `t5=35`.

1. Bestimmen Sie Zielwort und effektive Schiebeweite beziehungsweise Füllbits. (4 P)

```
srl  t2, t0, t1
sra  t3, t0, t1
sll  t4, t0, t5
srai t6, t0, 3
```

2. Welches Ergebnis liefert die dritte Zeile bei `t5=66`? (1 P)
3. Ist `slli t4, t0, 33` zulässig? Unterscheiden Sie die direkte von der Registerweite. (2 P)
4. Vergleichen Sie `sra` von −11 um zwei Stellen mit einer mathematischen Division durch vier und Abschneiden zur null. (1 P)

Aufgabe 4: Direkte Konstanten (8 Punkte)

1. Ordnen Sie zu: native Instruktion, Pseudoinstruktion oder unzulässiger Operand. Begründen Sie kurz. (6 P)

```
addi t0, x0, -2047
addi t0, x0, -2049
andi t0, t1, 2046
andi t0, t1, 4094
li   t0, 3000
xori t0, t1, -2
```

2. Stellen Sie −19 als zwölf Bits, drei Hexziffern und vorzeichenerweitertes 32-Bit-Wort dar. (1 P)
3. Ersetzen Sie das Subtrahieren von 37 durch `addi`. Warum funktioniert dasselbe Ein-Befehl-Schema beim Subtrahieren von −2050 nicht? (1 P)

Aufgabe 5: Registerbefehle codieren (14 Punkte)

R-Format von Bit 31 nach 0: funct7 (7), rs2 (5), rs1 (5), funct3 (3), rd (5), opcode (7 Bit). Für add: funct7 = `0000000`, funct3 = `000`, opcode = `0110011`. Für sub: funct7 = `0100000`, übrige Funktionswerte gleich.

1. Codieren Sie `sub x12, x14, x9`: Felder, 32-Bit-Muster und Hexwort. (6 P)
2. Decodieren Sie `0x012406B3`: Felder und Befehl. (6 P)
3. Ändert ein neuer Inhalt von x14 das Wort aus Teil 1? Genügt der Opcode allein zur Unterscheidung von add und sub? (2 P)

Aufgabe 6: Immediate-Wort und Byteadressen (12 Punkte)

I-Format: imm (12), rs1 (5), funct3 (3), rd (5), opcode (7 Bit). Für addi gilt funct3 = `000` und opcode = `0010011`. `a0=x10`, `a1=x11`.

1. Codieren Sie `addi a1, a0, -21` mit Feldern, Binärwort und Hexwort. (6 P)
2. Der folgende native Ausschnitt beginnt bei `0x2400`. Nennen Sie alle Befehlsadressen, die Adresse von `Pruefen`, den PC nach dem letzten Befehl und den Platzbedarf. Welche Konstante steht zuletzt in t3? (4 P)

```
lui  t3, 0x2
addi t3, t3, -1000
Pruefen:
add  t4, t3, x0
```

3. Geben Sie die Bytefolge des Worts aus Teil 1 ab `0x3400` in Little Endian an. (2 P)

Aufgabe 7: Eine Rechenvorschrift in Venus

Implementieren Sie $Y = (A - B) + (C \cdot 4)$ mit Schieben statt Multiplikationsbefehl. Eingaben in `t0`–`t2` bleiben erhalten; `t3` hält die Differenz, `t4` den verschobenen Wert und `t5` das Ergebnis.

1. Sagen Sie Zwischenwerte und Y für $(12, 19, 5)$, $(0, 0, 0)$, $(-8, 7, 2)$ und $(9, -4, -2)$ voraus.
2. Schreiben Sie ein vollständiges Venus-Programm mit Einstieg main und Prüfmarke unmittelbar vor der Ausgabe.
3. Prüfen Sie jede Variante nach Neustart. Venus-Kursvertrag: Druckdienst `a0=1`, Wert in `a1`; Abschluss `a0=10`, jeweils `ecall`.
4. Notieren Sie Vorhersage, Registeranzeige vor der Ausgabe und Konsolenausgabe. Untersuchen Sie außerdem die native Entwicklung von `li t0, 7192`.

Die neue Referenzdatei ist `beispiele/lab_02_messkorrektur.s`; vergleichen Sie erst nach Ihrer eigenen Implementierung.